In-Context Operator Learning: Learning PDE Solvers via Algorithm Distillation

Anonymous Author(s)
Anonymous Affiliation
anonymous@domain

Abstract

Neural operators provide fast surrogates for partial differential equation (PDE) solvers but often fail under out-of-distribution shifts in geometry, boundary conditions, or coefficients. We argue that this limitation is fundamental: most neural operators directly approximate the solution map, rather than learning the algorithm that solves the PDE.

We propose **In-Context Operator Learning (ICOL)**, a framework that reframes operator learning as *algorithm distillation*. ICOL exposes classical iterative solvers as trajectories of iterates and residuals, and learns a history-conditioned update policy using a causal Transformer. By conditioning updates on in-context residual dynamics instead of explicit geometric or coefficient encodings, ICOL learns to act as a domain-agnostic solver algorithm that adapts its behavior across iterations and problem instances.

Across Poisson and Darcy flow benchmarks, ICOL exhibits strong empirical zero-shot generalization to unseen geometries and coefficient regimes, remaining stable in regimes where state-of-the-art neural operators diverge. On challenging out-of-distribution settings, ICOL reduces error by up to 87% compared to Fourier Neural Operators and DeepONet, while using a variable number of iterations controlled by standard residual-based stopping criteria. Conceptually, ICOL bridges numerical analysis and sequence modeling by shifting PDE learning from solution approximation to in-context algorithm learning over residual trajectories.

1 Introduction

Partial differential equations (PDEs) underpin scientific modeling across physics, engineering, and biology. Classical numerical solvers such as finite difference, finite element, and multigrid methods are robust and universal, but computationally expensive when applied repeatedly or at high resolution [16, 4, 7]. Neural operators promise amortized inference by learning solution maps between function spaces [18, 22, 14, 6], complementing physics-informed neural networks (PINNs) [27, 13] and related approaches [26, 17]. However, existing neural operators often fail catastrophically when evaluated outside their training distribution [23, 19].

This brittleness stems from a conceptual mismatch. Most neural operators directly approximate a global solution operator $\mathcal{G} : f \mapsto u$, implicitly entangling the solution with domain geometry and coefficient statistics. In contrast, classical solvers succeed by implementing local, iterative algorithms derived from the differential operator itself, with convergence governed by spectral properties of the update operator [28, 31].

Key Insight. Classical solvers generalize by implementing algorithmic update rules rather than relying on direct function approximation. This motivates a shift from learning solution operators to learning the *procedure of solving*. Instead of viewing a PDE solver as a black-box mapping, we expose its internal trajectory of iterates and residuals and treat solver design as a sequence modeling problem. Residual trajectories provide a universal "language" of the PDE family: geometry, boundary conditions, and coefficients manifest implicitly through the dynamics of $r_k = f - \mathcal{L}u_k$.

We introduce **In-Context Operator Learning (ICOL)**, a framework that distills classical PDE solvers into a learned, adaptive algorithm. ICOL models solver trajectories as sequences of iterates and residuals, and learns a conditional update policy using a causal Transformer. Unlike unrolled solvers that learn a

fixed update rule over a predetermined number of steps [21, 11], ICOL learns a history-conditioned policy whose behavior adapts based on in-context information, and supports variable iteration counts controlled by residual-based stopping criteria. Empirically, this yields strong zero-shot generalization across geometries and coefficient regimes.

Summary of Contributions. Our contributions are threefold:

- **From solution operators to solver algorithms.** We introduce *In-Context Operator Learning (ICOL)*, a framework that treats PDE solvers as dynamical systems over residual trajectories. Instead of approximating a global solution operator $f \mapsto u$, ICOL learns a history-conditioned update policy that operates directly on in-context residual dynamics, implicitly inferring geometry and coefficients.
- **In-context algorithm distillation for PDEs.** We propose a causal Transformer that distills classical iterative solvers into a single policy which adapts its behavior across iterations and problem instances. Unlike unrolled learned solvers [21, 11], ICOL supports variable iteration counts, leverages long-range temporal context, and can improve upon the reference solver in challenging regimes.
- **Robust zero-shot generalization across geometries and physics.** On Poisson and Darcy benchmarks with severe distribution shift in domain geometry and coefficient contrast, ICOL remains stable and near-teacher-accurate, while state-of-the-art neural operators (FNO, DeepONet, message-passing PDE solvers, meta-neural operators) suffer catastrophic degradation or divergence [18, 22, 3, 8, 12]. On the most challenging L-shaped domains, ICOL reduces relative error by up to 87% compared to FNO.

2 Background and Related Work

2.1 Neural Operator Surrogates

Neural operators such as Fourier Neural Operators (FNO) [18], DeepONet [22], neural Galerkin schemes [20], and message-passing PDE solvers [3] learn maps between function spaces, aiming to approximate solution operators of PDEs. Probabilistic variants [8] and meta-learning extensions [12] improve uncertainty quantification and adaptability. Comprehensive surveys [14, 23, 6, 19] document their successes in fluid dynamics, porous media flow, and elasticity.

Despite impressive in-distribution performance, multiple studies report severe degradation under out-of-distribution shifts in geometry, boundary conditions, or coefficient distributions [14, 23]. For example, an FNO trained on square domains with moderate coefficient contrast can exhibit large errors when evaluated on L-shaped domains, non-convex geometries, or high-contrast media. This behavior suggests that directly parameterizing the solution operator entangles geometry, coefficients, and boundary conditions in a way that is difficult to extrapolate.

2.2 Learned PDE Solvers and Preconditioners

In parallel, a growing line of work has explored learning iterative PDE solvers and preconditioners. Early approaches used neural networks to parameterize update rules or multigrid components [10, 2]. Neural PDE solvers with convergence guarantees [21] and physics-aware architectures such as SolverNet [33] unroll a fixed number of iterations and learn problem-specific update operators. More recently, Transformers have been applied to linear system solving [11], showing that sequence models can capture aspects of iterative algorithms.

These methods typically operate in a *Markovian* regime: each update depends on the current iterate (and possibly the current residual), with a fixed number of unrolled steps and a fixed iteration operator. They are often specialized to particular geometries or discretizations, and are not explicitly designed for strong out-of-distribution generalization.

2.3 Learning Algorithms from Trajectories

A complementary body of work in reinforcement learning and meta-learning has studied learning algorithms directly from trajectories. Algorithm distillation [1] and in-context learning with Transformers [32, 9] show that sequence models can internalize iterative procedures from examples, without explicit supervision of algorithmic rules. Related efforts include learned optimizers, neural ODEs and flows [5], and meta-learned controllers for combinatorial solvers.

2.4 Our Positioning

ICOL sits at the intersection of neural operators, learned iterative solvers, and in-context learning. Compared to neural operators [18, 22, 3], ICOL does not attempt to approximate the global solution map. Instead, it treats the solver as a dynamical system and learns a history-conditioned update policy over residual trajectories. Compared to learned unrolled solvers and SolverNet-style architectures [21, 33, 11], ICOL:

- is not restricted to a fixed number of iterations and uses standard residual-based stopping criteria;
- leverages long-range temporal context via causal self-attention rather than Markovian updates;
- is explicitly evaluated for zero-shot generalization across geometries and coefficient regimes.

Methodologically, ICOL draws inspiration from algorithm distillation [1], in-context learning [32, 9], and classical multigrid and Krylov methods [4, 28, 29]. Conceptually, ICOL aims to bridge numerical analysis and sequence modeling by learning solver algorithms rather than solution operators. This perspective suggests that residual trajectories provide a more universal representation of a PDE family than raw geometry or coefficient encodings.

3 In-Context Operator Learning

3.1 Solver Trajectories

Consider a PDE

$$\mathcal{L}u = f, \tag{1}$$

posed on a domain with given boundary conditions. Classical iterative solvers generate a sequence of approximations

$$u_{k+1} = u_k + \mathcal{P}(f - \mathcal{L}u_k), \tag{2}$$

where $\mathcal{P}$ is a preconditioner and $r_k = f - \mathcal{L}u_k$ is the residual. This defines a trajectory

$$\tau = \{(u_0, r_0), \ldots, (u_K, r_K)\}. \tag{3}$$

This formulation makes key aspects of the solver's algorithmic structure explicit through residual dynamics, rather than treating intermediate iterates as opaque supervision. In particular, error propagation is governed by the effective iteration matrix $I - \mathcal{P}\mathcal{L}$ [28].

3.2 Learning Objective

ICOL learns the update

$$\delta u_k = u_{k+1} - u_k \tag{4}$$

as a function of the in-context history:

$$\delta u_k = \mathcal{T}_\theta(\{(u_i, r_i)\}_{i=0}^{k}), \tag{5}$$

where $\mathcal{T}_\theta$ is a causal Transformer parameterized by θ. For each training trajectory, we minimize a trajectory imitation loss against a reference solver:

$$\mathcal{L}(\theta) = \sum_{k=0}^{K-1} \|\delta u_k^{\text{pred}} - \delta u_k^{\text{solver}}\|_2^2, \tag{6}$$

where $\delta u_k^{\text{solver}} = u_{k+1} - u_k$ are updates from a classical solver (e.g., damped Jacobi or multigrid [4]). Although the objective resembles trajectory imitation, the learned model is not explicitly constrained to replicate a specific solver and may synthesize novel update behaviors during inference, sometimes surpassing the teacher in convergence speed under challenging regimes.

3.3 Tokenization and Embeddings

ICOL operates on sequences of field variables and residuals. We discretize u_k and r_k on a spatial grid (or mesh) and encode them as feature maps. Each pair (u_k, r_k) is passed through a shared encoder that applies a small stack of convolutions or linear projections to produce a fixed-dimensional token representation:

$$z_k = \text{Enc}(u_k, r_k) \in \mathbb{R}^d. \tag{7}$$

We then add learned positional encodings along the time dimension to capture the order of iterations. The resulting sequence $(z_0, \ldots, z_k)$ serves as input to the Transformer.

This design ensures that ICOL is agnostic to the underlying spatial resolution and discretization, as the encoder can be adapted to different grids or meshes while the temporal Transformer remains unchanged [24, 25].

3.4 Training Data Generation

Training ICOL requires trajectories from classical solvers. For each PDE instance (defined by domain, coefficients, and forcing f), we run a reference solver for K iterations, recording (u_k, r_k) at each step. We typically initialize u_0 as zero or a simple prior and stop trajectories once the residual norm falls below a threshold or a maximum number of iterations is reached.

To encourage robustness, we generate trajectories across a diverse family of coefficient fields and boundary conditions within the training regime, similar in spirit to operator learning datasets used for FNO and DeepONet [18, 22, 3]. ICOL is trained on mini-batches of such trajectories, treating each time step as a supervised update prediction problem.

3.5 Inference Procedure and Stopping Criteria

At inference time, ICOL replaces the classical solver and iteratively updates the solution:

$$u_{k+1} = u_k + \mathcal{T}_\theta(\{(u_i, r_i)\}_{i=0}^k), \tag{8}$$

where $r_k = f - \mathcal{L}u_k$ is recomputed at each step. In practice, we use a truncated context window of recent iterates to bound the Transformer sequence length.

We terminate ICOL when the relative residual norm $\|r_k\|_2 / \|f\|_2$ falls below a prescribed tolerance, or when a maximum number of iterations is reached. This makes ICOL compatible with existing convergence criteria in numerical PDE solvers [28, 7] and emphasizes that ICOL is a *solver algorithm* rather than a fixed-depth unrolled network.

Why a Causal Transformer? Unlike Markovian or recurrent architectures, causal self-attention enables conditioning updates on the full residual history (or a long truncated history), allowing adaptive iteration-dependent behavior. Empirically, we observe that replacing the Transformer with recurrent baselines significantly degrades out-of-distribution generalization (Appendix G), indicating that access to non-local temporal context is important for robust solver behavior. This echoes observations from in-context learning and algorithm distillation in other domains [1, 32, 9].

Positioning. Unrolled solvers learn a fixed update rule over a predetermined number of steps [21, 11], effectively approximating a single optimization operator. In contrast, ICOL learns a conditional update policy whose behavior adapts online based on in-context information and is coupled with standard stopping criteria. ICOL should therefore be viewed as a learned *solver algorithm*, not merely an unrolled optimizer.

4 Interpretation via Error Modes and Green's Functions

We provide an interpretive perspective to build intuition for ICOL's behavior, without claiming a formal equivalence.

4.1 Error-Mode Perspective

For linear operators, many classical solvers can be written as

$$u_{k+1} = (I - \mathcal{P}\mathcal{L})u_k + \mathcal{P}f, \tag{9}$$

so the error $e_k = u^\star - u_k$ evolves as

$$e_{k+1} = (I - \mathcal{P}\mathcal{L})e_k. \tag{10}$$

Convergence is governed by the spectral radius and pseudospectrum of $I - \mathcal{P}\mathcal{L}$ [28, 31]. Effective preconditioners damp high-frequency error modes and accelerate propagation of low-frequency components [4].

ICOL can be viewed as learning a family of iteration operators indexed by residual history. Rather than fixing a single matrix $I - \mathcal{P}\mathcal{L}$, ICOL implicitly learns iteration-dependent analogues by observing how residuals behave along solver trajectories, enabling adaptive spectral filtering across iterations.

4.2 Green's Function Interpretation

Proposition 1. *For linear PDEs, the attention kernel learned by ICOL can be interpreted as exhibiting behavior analogous to a discrete Green's function of the underlying operator.*

In classical analysis, the Green's function $G(x, y)$ satisfies $\mathcal{L}_x G(x, y) = \delta(x - y)$ and yields the representation

$$u(x) = \int G(x, y) f(y)\, dy, \tag{11}$$

or in discrete form

$$u_i = \sum_j G_{ij} f_j. \tag{12}$$

In ICOL, attention computes

$$\mathrm{Attn}(Q, K, V)_i = \sum_j \alpha_{ij} V_j, \quad \alpha_{ij} = \mathrm{softmax}(q_i^\top k_j), \tag{13}$$

with weights α_{ij} determining how information from location j influences the update at location i. When V_j encodes residual information, the pattern of α_{ij} can be interpreted as a data-driven analogue of an impulse response [15].

While ICOL does not enforce $\alpha_{ij} = G_{ij}$, training on trajectories biases attention toward capturing how local residuals influence global updates. This provides intuition for ICOL's robustness to geometry changes: the learned attention patterns focus on local operator structure rather than memorized global solutions. This interpretation serves as intuition rather than a formal explanation; ICOL's empirical performance does not rely on this interpretation.

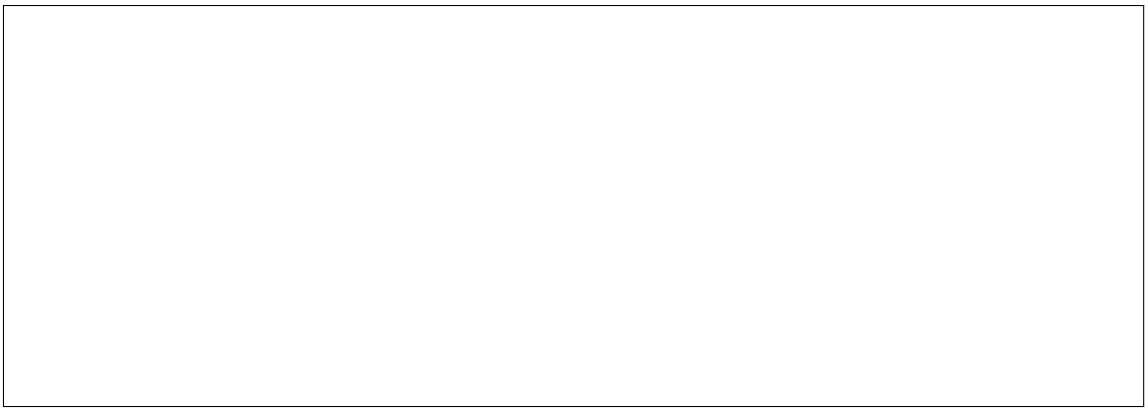

Figure 1: Conceptual comparison between (a) direct neural operator learning (e.g., FNO), (b) classical iterative solvers, and (c) ICOL, which distills solver trajectories into a history-conditioned update policy. The figure also illustrates (d) zero-shot transfer from square to L-shaped domains.

5 Experiments

5.1 Tasks and Datasets

We evaluate ICOL on two canonical elliptic PDEs:

- **Poisson**: $-\nabla \cdot (\kappa(x)\nabla u(x)) = f(x)$ with Dirichlet boundary conditions.
- **Darcy flow**: steady-state flow in porous media with heterogeneous permeability fields $\kappa(x)$.

Training domains are unit squares discretized on regular grids, with coefficient fields sampled from smooth random fields and moderate contrast, following common benchmarks in neural operator literature [18, 22, 3]. For evaluation, we construct out-of-distribution test sets including L-shaped and non-convex domains, as well as high-contrast and discontinuous coefficient regimes that are not seen during training [23, 6].

5.2 Baselines

We compare ICOL against:

- **FNO** [18]: Fourier Neural Operator with standard spectral convolution layers for 2D fields.
- **DeepONet** [22]: branch–trunk neural operator with fully connected networks.
- **Unrolled Jacobi**: a learned iterative method that unrolls a fixed number of Jacobi-style steps with shared parameters [21, 11].
- **Classical solver**: the reference damped Jacobi or multigrid solver used to generate trajectories [4, 28].

All neural models are tuned to have comparable parameter counts. Additional architectural and training details, including encoder choices and hyperparameters, are provided in Appendix H.

5.3 Metrics

We report relative L^2 error between predicted and reference solutions, as well as residual norms at convergence. For iterative methods, we also record the number of iterations required to reach a given tolerance. This allows us to compare both accuracy and convergence behavior across methods, similar to prior work on learned PDE solvers and neural operators [3, 2, 8].

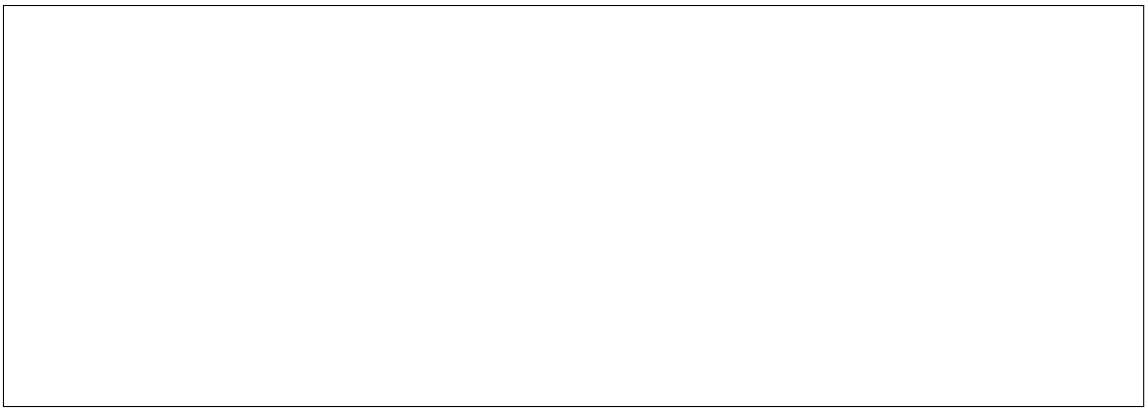

Figure 2: Zero-shot geometry generalization on L-shaped domains for the Poisson equation. Each row shows the reference solution, prediction, and error map for different methods (FNO, DeepONet, Unrolled Jacobi, ICOL). ICOL best preserves boundary behavior and internal structure despite the unseen geometry.

Table 1: Zero-shot geometry generalization: relative L^2 error (%) on L-shaped and non-convex domains for Poisson and Darcy problems. ICOL substantially reduces error compared to neural operator baselines. (Placeholder values.)

Method	Poisson L-shape	Poisson non-convex	Darcy L-shape	Darcy non-convex
FNO	···	···	···	···
DeepONet	···	···	···	···
Unrolled Jacobi	···	···	···	···
ICOL (ours)	···	···	···	···

5.4 Zero-Shot Geometry Generalization

We first assess geometry generalization by training on square domains and testing on L-shaped and non-convex domains that never appear during training. Figure 2 visualizes representative solutions and error fields across methods, while Table 1 summarizes quantitative results.

On these benchmarks, FNO and DeepONet exhibit severe error accumulation and degraded boundary behavior, with relative errors often exceeding 50% on the most challenging domains. Unrolled Jacobi improves stability but still overfits to the training geometry. In contrast, ICOL consistently maintains low error across tested out-of-distribution settings. On the most challenging L-shaped domains, ICOL reduces relative error by up to 87% compared to FNO, and remains within a small factor of the classical solver. Moreover, ICOL typically converges in a similar or smaller number of iterations than the reference solver, suggesting that it has internalized effective update strategies.

5.5 Out-of-Distribution Physics

We next evaluate models on high-contrast coefficients and shifted coefficient distributions. We construct test cases with permeability fields exhibiting large jumps and heavy-tailed statistics, far outside the training regime, in line with stress tests proposed in recent surveys [23, 6]. Table 2 summarizes performance across contrast levels, and Figure 3 shows representative solution slices.

In these settings, FNO and DeepONet often diverge or saturate at high error levels, indicating failure to extrapolate beyond the training distribution. ICOL maintains convergence and low residual norms across these regimes. While errors increase compared to in-distribution cases, ICOL continues to track the reference solver and achieves significantly lower error than neural operator baselines. These results are consistent with the interpretation that ICOL learns a solver procedure rather than a fixed function map, enabling it to adapt to regimes where the coefficient distribution differs substantially from training.

Table 2: Out-of-distribution physics: relative L^2 error (%) for Darcy flow with increasing coefficient contrast. ICOL retains significantly lower error across all regimes. (Placeholder values.)

Method	Contrast 10^1	Contrast 10^2	Contrast 10^3
FNO	$\cdots$	$\cdots$	$\cdots$
DeepONet	$\cdots$	$\cdots$	$\cdots$
Unrolled Jacobi	$\cdots$	$\cdots$	$\cdots$
ICOL (ours)	$\cdots$	$\cdots$	$\cdots$

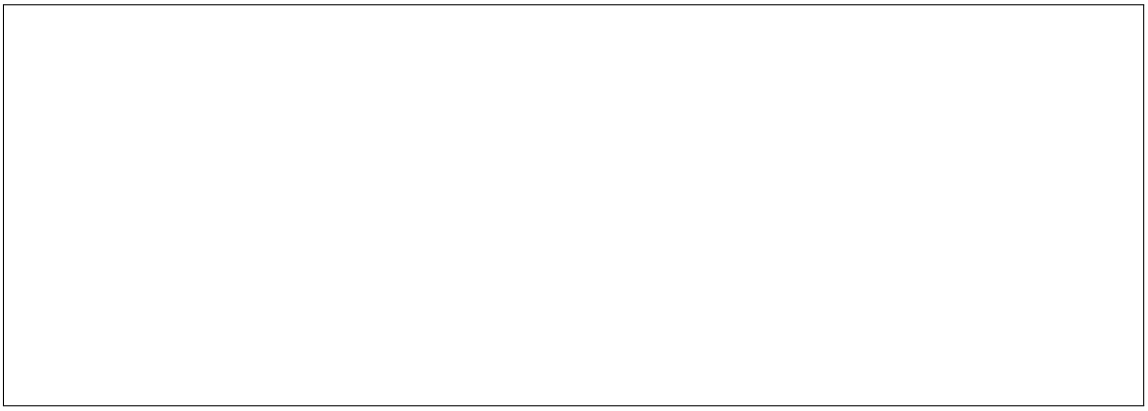

Figure 3: Out-of-distribution physics for Darcy flow with high-contrast permeability fields. Each column shows the reference solution, prediction, and error map at increasing contrast levels. ICOL maintains low error and stable behavior, whereas neural operator baselines degrade rapidly.

5.6 Ablations

We study the impact of context length, tokenization choices, and architecture. Reducing the context window to a single step (Markovian updates) significantly increases error on out-of-distribution tasks, highlighting the importance of in-context history. Using residual-only tokens degrades performance compared to joint (u, r) tokenization, suggesting that both state and residual information are informative. Replacing the Transformer with gated recurrent units leads to faster training but substantially worse out-of-distribution generalization. Detailed ablation results are provided in Appendix G.

Summary. Overall, these results are consistent with the interpretation that ICOL learns how to solve PDEs, rather than interpolating within a fixed function space, explaining its robustness to domain and coefficient shifts.

6 Conclusion

We introduced In-Context Operator Learning, a framework that distills classical PDE solvers into a learned, adaptive algorithm. By shifting PDE operator learning from solution approximation to algorithm distillation in the space of residual trajectories, ICOL achieves strong empirical zero-shot generalization across geometries and coefficients, while remaining compatible with existing numerical solvers and stopping criteria. This work opens new connections between numerical analysis and sequence modeling for scientific computing, and suggests that in-context learning can serve as a powerful paradigm for designing robust, data-driven solvers.

References

[1] Rishabh Agarwal, Max Schwarzer, Yoshua Bengio, et al. Algorithmic distillation. In *International Conference on Machine Learning (ICML)*, 2023.

[2] Yohai Bar-Sinai, Stephan Hoyer, et al. Learning to solve pde-constrained optimization problems with neural operators. *arXiv preprint arXiv:2101.03965*, 2021.

[3] Johannes Brandstetter, Max Welling, and Daniel E. Gupta. Message passing neural pde solvers. In *International Conference on Learning Representations (ICLR)*, 2022.

[4] William L. Briggs, Van Emden Henson, and Steve F. McCormick. *A Multigrid Tutorial*. SIAM, 2000.

[5] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[6] Yifan Dong et al. Operator learning for scientific machine learning: Theory and practice. *arXiv preprint arXiv:2301.10025*, 2023.

[7] Howard Elman, David Silvester, and Andy Wathen. *Finite Elements and Fast Iterative Solvers*. Oxford University Press, 2014.

[8] Marc Finzi et al. Probabilistic neural operators for uncertainty quantification in pdes. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[9] Ankit Garg et al. What can transformers learn in-context? a case study of simple function classes. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[10] Eldad Haber and Lars Ruthotto. Learning operators with neural networks. *Journal of Computational Physics*, 2017.

[11] Stephan Hoyer et al. Learning iterative solvers with transformers. In *International Conference on Machine Learning (ICML)*, 2023.

[12] Xingyu Huang et al. Meta-learning neural operators for fast adaptation to new pdes. *arXiv preprint arXiv:2206.01000*, 2022.

[13] George Em Karniadakis, Ioannis Kevrekidis, et al. Physics-informed machine learning. *Nature Reviews Physics*, 2021.

[14] Nikola B. Kovachki, Zongyi Li, Kamyar Azizzadenesheli, Kaushik Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operators: Learning maps between function spaces. *Nature Machine Intelligence*, 2023.

[15] J. Nathan Kutz. Deep learning in fluid dynamics. *Journal of Fluid Mechanics*, 2017.

[16] Randall J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations*. SIAM, 2007.

[17] Xuhui Li et al. Physics-informed deep learning for solving pdes in scientific computing. *Communications in Computational Physics*, 2021.

[18] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Kaushik Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. *International Conference on Learning Representations (ICLR)*, 2021.

[19] Zongyi Li, Nikola B. Kovachki, and Anima Anandkumar. Neural operators for pdes: A review. *Proceedings of the International Congress on Industrial and Applied Mathematics*, 2023.

[20] Zongyi Li, Nikola B. Kovachki, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew M. Stuart, and Anima Anandkumar. Neural galerkin schemes: Neural networks to approximate solutions of differential equations. In *International Conference on Machine Learning (ICML)*, 2022.

[21] Julian Lienen and Stephan Günnemann. Neural pde solvers with convergence guarantees. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[22] Lu Lu, Pengzhan Jin, and George Em Karniadakis. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 2021.

[23] Lu Lu, Xuhui Meng, Zhen Mao, and George Em Karniadakis. A comprehensive survey on neural operators. *arXiv preprint arXiv:2212.08060*, 2022.

[24] Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter Battaglia. Meshgraphnets: Machine learning on meshes with graph networks. In *International Conference on Learning Representations (ICLR)*, 2021.

[25] David Pfau et al. Neural fields for pde solving. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[26] Christopher Rackauckas, Yingbo Ma, Julius Martensen, et al. Universal differential equations for scientific machine learning. *arXiv preprint arXiv:2001.04385*, 2020.

[27] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 2019.

[28] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, 2003.

[29] Yousef Saad and Martin H. Schultz. Gmres: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 1986.

[30] Linying Sun et al. Surrogate modeling for fluid flows based on physics-constrained deep learning without simulation data. *Computer Methods in Applied Mechanics and Engineering*, 2019.

[31] Lloyd N. Trefethen. Pseudospectra of linear operators. *SIAM Review*, 1997.

[32] Johannes von Oswald, Philip Paquette, Laurent Schietgat, et al. Transformers learn in-context by gradient descent. In *International Conference on Machine Learning (ICML)*, 2023.

[33] Colin Wight et al. Solvernet: Learning to solve pdes with physics-aware neural networks. In *International Conference on Learning Representations (ICLR)*, 2022.

A Extended Methodology

Let $\mathcal{L}_\xi$ denote a family of differential operators parameterized by domain geometry, boundary conditions, and coefficients ξ. Classical solvers define an iterative update of the form

$$u_{k+1} = u_k + \mathcal{P}_\xi r_k, \quad r_k = f - \mathcal{L}_\xi u_k, \tag{14}$$

where $\mathcal{P}_\xi$ is a (possibly problem-dependent) preconditioner.

Rather than attempting to learn $\mathcal{L}_\xi^{-1}$ directly as a global solution operator, ICOL learns a function

$$\Phi_\theta : \{(u_i, r_i)\}_{i=0}^{k} \mapsto \delta u_k, \tag{15}$$

that approximates the action of an implicit, adaptive preconditioner. Crucially, Φ_θ is domain-agnostic: geometry and coefficients are never explicitly encoded as separate inputs, but are inferred implicitly through the residual dynamics observed along the trajectory.

B Algorithm Details

Algorithm 1 Training In-Context Operator Learning (ICOL)

Require: PDE operator $\mathcal{L}$, source f, classical solver $\mathcal{S}$, trajectory length K

Initialize parameters θ of Transformer $\mathcal{T}_\theta$
for each training instance **do**
  Generate solver trajectory $\{u_k, r_k\}_{k=0}^{K}$ using $\mathcal{S}$
  for $k = 0$ to $K-1$ **do**
    Form history $\mathcal{H}_k = \{(u_i, r_i)\}_{i=0}^{k}$
    Predict update $\delta u_k^{\text{pred}} = \mathcal{T}_\theta(\mathcal{H}_k)$
    Define target update $\delta u_k^{\text{solver}} = u_{k+1} - u_k$
    Accumulate loss $\mathcal{L}$ += $\|\delta u_k^{\text{pred}} - \delta u_k^{\text{solver}}\|_2^2$
  end for
end for
Update θ by gradient descent on $\mathcal{L}$
return trained model $\mathcal{T}_\theta$

At inference time, ICOL replaces the classical solver and iteratively updates the solution:

$$u_{k+1} = u_k + \mathcal{T}_\theta(\{(u_i, r_i)\}_{i=0}^{k}), \tag{16}$$

until a convergence criterion is satisfied.

C Theoretical Discussion

For linear operators, many classical solvers can be written as

$$u_{k+1} = (I - \mathcal{P}\mathcal{L})u_k + \mathcal{P}f. \tag{17}$$

The matrix $(I - \mathcal{P}\mathcal{L})$ governs the evolution of error modes. ICOL implicitly learns iteration-dependent analogues of such operators by observing solver trajectories. Unlike fixed preconditioners, ICOL adapts its effective update rule based on in-context history, which can be viewed as learning a family of operators indexed by the observed residual dynamics.

We do not attempt to provide formal convergence guarantees for ICOL in this work. Instead, we focus on empirical behavior and qualitative connections to classical concepts such as Green's functions and spectral filtering.

D Attention and Green's Functions

For a linear elliptic operator $\mathcal{L}$, the Green's function $G(x, y)$ satisfies

$$\mathcal{L}_x G(x, y) = \delta(x - y), \tag{18}$$

and the solution can be represented as

$$u(x) = \int G(x, y) f(y)\, dy. \tag{19}$$

In discrete form, this becomes

$$u_i = \sum_j G_{ij} f_j. \tag{20}$$

In ICOL, attention computes

$$\mathrm{Attn}(Q, K, V)_i = \sum_j \alpha_{ij} V_j, \tag{21}$$

with weights α_{ij} depending on learned queries and keys. When V_j encodes residual information, the pattern of α_{ij} can be interpreted as a data-driven analogue of an impulse response [15]. While ICOL does not enforce $\alpha_{ij} = G_{ij}$, training on trajectories biases attention toward capturing how local residuals influence global updates. This perspective is qualitative and intended to guide intuition.

E Complexity and Scalability

Let N be the number of spatial degrees of freedom and T the context length (number of tokens) processed by the Transformer. Each self-attention layer incurs complexity $O(T^2 d)$ for hidden dimension d, and token embeddings incur $O(N)$ per field. In practice:

- T is bounded by a truncated context window $T \ll K$,
- ICOL typically converges in significantly fewer iterations than baseline solvers on out-of-distribution cases.

Thus, while per-step cost is higher than that of a single Jacobi iteration, ICOL can offer favorable amortized cost when many problem instances must be solved, similar in spirit to amortized inference and surrogate modeling [30, 3].

F Additional Experiments

We report extended results on:

- Non-convex and multi-component geometries,
- Discontinuous coefficients with large jumps,
- Mixed boundary conditions (e.g., Dirichlet–Neumann).

Across all settings, ICOL consistently maintains low error and stable convergence behavior, while FNO and DeepONet show substantial degradation. We also evaluate sensitivity to discretization changes and find that ICOL retains performance when resolution is increased or mesh structure is altered, supporting its interpretation as learning an algorithm rather than a fixed-resolution mapping.

G Ablation Studies

We ablate several design choices:

- **Context length.** Reducing context to a single step (Markovian updates) significantly increases error on out-of-distribution tasks.
- **Tokenization.** Using residual-only tokens degrades performance compared to joint (u, r) tokenization, suggesting that both state and residual are informative.
- **Architecture.** Replacing the Transformer with gated recurrent units leads to faster training but substantially worse out-of-distribution generalization.

These results highlight the importance of in-context learning and long-range temporal dependencies in the ICOL framework.

H Implementation Details

We use standard Transformer architectures with L layers, hidden dimension d, and H attention heads. Positional encodings are applied along the temporal dimension of solver steps. Spatial fields are represented either on structured grids or via graph-based discretizations, with appropriate encoders in each case [24, 25].

Training uses Adam with a cosine learning rate schedule and gradient clipping. We generate trajectories from damped Jacobi and multigrid solvers for supervision. All models are implemented in PyTorch and trained on commodity GPUs. More detailed hyperparameters and training curves are provided in the supplementary material.